```
import numpy as np
import pandas as pd

ds = pd.read_csv("wan_dataset.csv")
```

1. Min-Max Normalization

```
x = ds['latency_ms']

normalized = (x - x.min()) / (x.max() - x.min())

print(normalized.head())
```

```
0    1.000000
1    0.766667
2    0.266667
3    0.000000
4    0.166667
Name: latency_ms, dtype: float64
```

2. Z-Score Normalization

```
x = ds['latency_ms']

normalized = (x - x.mean()) / x.std()

print(normalized.head())
```

```
0    1.521796
1    0.858436
2   -0.563051
3   -1.321177
4   -0.847348
Name: latency_ms, dtype: float64
```

3. Decimal Scaling

```
x = ds['latency_ms']

j = len(str(int(x.abs().max())))

normalized = x / (10 ** j)

print(normalized.head())
```

```
0    0.35
1    0.28
2    0.13
3    0.05
4    0.10
Name: latency_ms, dtype: float64
```

4. Comparison of Normalization Methods

```
result = pd.DataFrame()

result['Original'] = ds['latency_ms']
result['Min-Max'] = (ds['latency_ms'] - ds['latency_ms'].min()) / (ds['latency_ms'].max() - ds['latency_ms'].min())
result['Z-Score'] = (ds['latency_ms'] - ds['latency_ms'].mean()) / ds['latency_ms'].std()

j = len(str(int(ds['latency_ms'].abs().max())))
result['Decimal Scaling'] = ds['latency_ms'] / (10 ** j)
```

```
print(result.head())
```

```
   Original   Min-Max   Z-Score  Decimal Scaling
0        35  1.000000  1.521796             0.35
1        28  0.766667  0.858436             0.28
2        13  0.266667 -0.563051             0.13
3         5  0.000000 -1.321177             0.05
4        10  0.166667 -0.847348             0.10
```

5. Discretization using Three Clusters

```
x = ds['latency_ms'].to_numpy()

centroids = np.array([x.min(), x.mean(), x.max()])

for i in range(100):
    distances = np.abs(x[:, None] - centroids)
    clusters = np.argmin(distances, axis=1)

    new_centroids = np.array([
        x[clusters == j].mean() if np.any(clusters == j) else centroids[j]
        for j in range(3)
    ])

    if np.allclose(centroids, new_centroids):
        break

    centroids = new_centroids

result = ds[['latency_ms']].copy()
result['Cluster'] = clusters

print("Centroids:", centroids)
print(result.head(20))
```

```
Centroids: [ 7.69035533 19.55576923 31.29190751]
    latency_ms  Cluster
0           35        2
1           28        2
2           13        0
3            5        0
4           10        0
5           24        1
6           34        2
7           31        2
8           17        1
9            6        0
10          34        2
11          27        2
12          13        0
13           5        0
14          10        0
15          24        1
16          34        2
17          30        2
18          17        1
19           6        0
```

6. Display the Range of Each Cluster

```
for i in range(3):
    values = result[result['Cluster'] == i]['latency_ms']

    if len(values) > 0:
        print("Cluster", i)
        print("Minimum:", values.min())
        print("Maximum:", values.max())
```

```
Cluster 0
Minimum: 5
Maximum: 13
Cluster 1
```

```
Minimum: 14
Maximum: 25
Cluster 2
Minimum: 26
Maximum: 35
```